```
    after_save  encrypter
    after_find  encrypter
  protected
    def after_find
    end
  end
```

We create a new `Encrypter` object and hook it up to the events `before_save`, `after_save`, and `after_find`. This way, just before an order is saved, the method `before_save` in the encrypter will be invoked, and so on.

So why do we define an empty `after_find` method? Remember that we said that for performance reasons `after_find` and `after_initialize` are treated specially. One of the consequences of this special treatment is that Active Record won't know to call an `after_find` handler unless it sees an actual `after_find` method in the model class. We have to define an empty placeholder to get `after_find` processing to take place.

This is all very well, but every model class that wants to use our encryption handler would need to include some eight lines of code, just as we did with our `Order` class. We can do better than that. We'll define a helper method that does all the work and make that helper available to all Active Record models. To do that, we'll add it to the `ApplicationRecord` class:

rails80/e1/ar/encrypter.rb

```
  class ApplicationRecord < ActiveRecord::Base
    self.abstract_class = true

    def self.encrypt(*attr_names)
      encrypter = Encrypter.new(attr_names)

      before_save encrypter
      after_save  encrypter
      after_find  encrypter

      define_method(:after_find) { }
    end
```